

ACADEMIA DE STUDII ECONOMICE DIN BUCUREȘTI
Facultatea de Cibernetică, Statistică și Informatică Economică
Specializarea Informatică Economică



Proiect Pachete Software

Studiu asupra sănătății studenților utilizând Python și SAS

Coordonator științific

Asistent univ. dr. ENE Gabriela

Studenți

MANEA Alina

MARINESCU Ștefania

București

2025

Cuprins

a) Definirea problemei.....	3
b) Informații necesare pentru rezolvare.....	3
Setul de date inițial.....	3
Variabilele din setul de date.....	3
c) Metode de calcul, algoritmi, formule de calcul utilizate.....	4
I) Python.....	4
II) SAS.....	28
d) Prezentarea rezultatelor și interpretarea economica a rezultatelor.....	48

a) Definirea problemei

În contextul actual al creșterii incidenței bolilor cronice în rândul populației tinere, identificarea timpurie a factorilor de risc devine esențială pentru prevenție și intervenție medicală precocă.

b) Informații necesare pentru rezolvare

Setul de date inițial

Setul de date reprezintă informații medicale colectate despre studenți, fiind structurat sub formă de tabel, unde fiecare rând corespunde unui individ. Coloanele conțin atât variabile demografice, cât și indicatori clinici. Setul de date este unul mare preluat de pe Kaggle, conținând 20 000 de observații (studenți) și 12 coloane (variabile) și poate fi găsit pe următorul link: <https://www.kaggle.com/datasets/slmsshk/medical-students-dataset/data>.

Variabilele din setul de date

- **Age (Tip: Numeric):** vârsta persoanei, exprimată în ani
- **Gender (Type: Categorical):** sexul biologic al persoanei (de exemplu: „Male”, „Female” sau lipsă).
- **Height (Type: Numeric):** înălțimea persoanei, exprimată în centimetri.
- **Weight (Type: Numeric):** greutatea persoanei, exprimată în kilograme.
- **Blood Type (Type: Categorical):** grupa sanguină a persoanei (ex: A, B, AB, O).
- **BMI (Type: Numeric):** indicele de masă corporală calculat pe baza înălțimii și greutății.
- **Temperature (Type: Numeric):** temperatura corporală a persoanei, exprimată în grade Fahrenheit.
- **Heart Rate (Type: Numeric):** frecvența cardiacă (numărul de bătăi pe minut).
- **Blood Pressure (Type: Numeric):** tensiunea arterială sistolică (valoarea superioară), exprimată în mmHg.
- **Cholesterol (Type: Numeric):** nivelul colesterolului din sânge, exprimat în mg/dL.

- **Diabetes (Type: Categorical):** indică dacă persoana are diabet („Yes” / „No”).
- **Smoking (Type: Categorical):** indică dacă persoana fumează („Yes” / „No”).

c) Metode de calcul, algoritmi, formule de calcul utilizate

I) Python

- **Citirea fișierului CSV și încărcarea datelor**

```
df_initial = pd.read_csv(file_path)
df = df_initial.copy()
```

Am încărcat fișierul CSV și am realizat și o copie pentru a păstra și datele originale.

- **Număr valori lipsă**

```
def missing_values_summary(df):

    missing = df.isna().sum()

    missing = missing[missing > 0]

    missing_df = pd.DataFrame({

        'Column': missing.index,

        'Missing Values': missing.values

    }).sort_values(by='Missing Values', ascending=False).reset_index(drop=True)

    return missing_df
```

Am verificat dacă există valori lipsă pe fiecare coloană și să fie returnat numărul lor, urmând să le tratăm ulterior și creează un nou DataFrame care conține numele coloanei și numărul valorilor lipse, fiind în același timp sortate descrescător.

- **Tratare valori lipsă**

```
def fill_missing_values(df):
    for col in df.columns:
        if df[col].isna().sum() > 0:
            if pd.api.types.is_numeric_dtype(df[col]):
                df[col].fillna(df[col].mean(), inplace=True)
            else:
                df[col].fillna(df[col].mode()[0], inplace=True)
    return df
```

Aceasta completează automat valorile lipsă în felul următor: pentru **coloane numerice**: înlocuiește NaN cu media (mean()), iar pentru **coloane categorice/text**: înlocuiește NaN cu cea mai frecventă valoare (mode()[0]). Rezultatul e un DataFrame curat, fără valori lipsă.

```
df_cleaned = fill_missing_values(df.copy())

df_cleaned['Age'] = df_cleaned['Age'].round(0).astype(int)
```

Important de menționat că în cazul valorilor lipsă înlocuite cu media ca număr întreg, am folosit o funcție pentru a o rotunji la cel mai apropiat număr întreg.

- **Codificarea binară a valorilor categorice în numerice**

```
df_cleaned['Diabetes'] = df_cleaned['Diabetes'].map({'Yes': 1, 'No': 0})
df_cleaned['Smoking'] = df_cleaned['Smoking'].map({'Yes': 1, 'No': 0})
df_cleaned['Gender'] = df_cleaned['Gender'].map({'Female': 0, 'Male': 1})
```

Pentru a integra variabilele categoriale în analize statistice, acestea au fost transformate în format numeric prin codificare. Variabilele binare precum Diabetes, Smoking și Gender au fost codificate manual: "Yes" și "Male" au fost înlocuite cu 1, iar "No" și "Female" cu 0.

- **Codificare One-Hot pentru valori categorice multiple**

```
categorical_cols = df_cleaned.select_dtypes(include=[object]).columns
```

```
df_encoded = pd.get_dummies(df_cleaned, columns=categorical_cols, drop_first=False, dtype=int)
```

Am codificat variabila categorică BloodType care avea mai multe valori („A”, „B”, „AB” și „O”) prin transformarea fiecărei valori unice într-o coloană binară separată, proces numit one-hot encoding. Această metodă creează patru coloane noi, câte una pentru fiecare grupă sanguină, iar în fiecare rând doar una dintre aceste coloane va avea valoarea 1, indicând grupa respectivă, iar restul vor fi 0. Astfel, variabilele textuale devin variabile numerice, potrivite pentru utilizarea în modele statistice și de învățare automată.

- **Metode de scalare**

```
#### StandardScaler
```

```
numeric_cols_to_scale = ['Height', 'Weight', 'BMI', 'Temperature', 'Heart Rate', 'Blood Pressure', 'Cholesterol']
```

```
scaler_standard = StandardScaler()
```

```
df_standardized = df_encoded.copy()
```

```
df_standardized[numeric_cols_to_scale] = scaler_standard.fit_transform(df_standardized[numeric_cols_to_scale])
```

```
##### MinMaxScaler
```

```
df_standardized[numeric_cols_to_scale] = df_standardized[numeric_cols_to_scale].round(2)
```

```

scaler_minmax = MinMaxScaler()

df_minmax = df_encoded.copy()

df_minmax[numeric_cols_to_scale] =
scaler_minmax.fit_transform(df_minmax[numeric_cols_to_scale])

df_minmax[numeric_cols_to_scale] = df_minmax[numeric_cols_to_scale].round(2)

```

Pentru că este important ca înainte de a folosi variabilele numerice în analize sau modele de învățare automată să le aducem pe o scară comună, deoarece ele pot avea unități și intervale foarte diferite, am aplicat metode de scalare.

Pentru a uniformiza variabilele numerice din setul de date, am aplicat două tipuri de scalare: StandardScaler și MinMaxScaler. StandardScaler transformă valorile astfel încât fiecare variabilă să aibă media 0 și deviația standard 1. Aceasta elimină diferențele de scară între variabile, facilitând analiza și funcționarea corectă a algoritmilor care presupun date distribuite normal sau sensibile la variația scalei, cum ar fi regresia sau metodele bazate pe distanță. MinMaxScaler redimensionează valorile astfel încât să fie cuprinse în intervalul [0,1], păstrând proporțiile relative dintre ele. Această scalare este utilă când modelele sau vizualizările necesită date într-un interval fix, fără valori negative sau foarte mari. Am aplicat aceste scalări pe variabilele numerice importante din setul nostru („Height”, „Weight”, „BMI”, „Temperature”, „Heart Rate”, „Blood Pressure”, „Cholesterol”) pentru a pregăti datele în vederea analizelor și modelării statistice.

- **Statistici descriptive**

```

stats = df_standardized.describe().round(2)

median_values = df_standardized.median(numeric_only=True).round(2)

sum_values = df_standardized.sum(numeric_only=True).round(2)

```

Aceste operații sunt utilizate pe datele curățate și standardizate. Variabilele stats, median_values și sum_values oferă o imagine de ansamblu asupra datelor numerice din df_standardized: stats cuprinde principalele statistici descriptive (media, abaterea standard, minimul, maximul etc.), median_values conține valorile mediane, iar sum_values totalurile

pentru fiecare coloană; toate rezultatele sunt rotunjite la două zecimale pentru o prezentare clară și concisă.

- **Interacțiuni coloane DataFrame**

```
if option == "Interacțiuni coloane DataFrame":

    st.write("### Interacțiuni cu coloanele (standardizat)")

    if "df" not in st.session_state:

        st.session_state.df = df_standardized.copy()

    df = st.session_state.df

    st.write("### DataFrame curent:")

    st.dataframe(df)

    col_names = list(df.columns)

    if col_names:

        selected_col = st.selectbox("Selectează o coloană:", col_names)

    else:

        st.warning("Nu există coloane disponibile în DataFrame.")

        selected_col = None

    if selected_col:

        st.subheader(f"Operații pentru coloana: **{selected_col}**")

        new_name = st.text_input("Introdu noul nume pentru coloană:", value=selected_col)

        if st.button("Redenumeste coloana"):

            if new_name and new_name != selected_col:

                st.session_state.df = st.session_state.df.rename(columns={selected_col: new_name})
```

```
st.success(f'Coloana '{selected_col}' a fost redenumită în '{new_name}!')
```

```
st.rerun()
```

else:

```
st.info("Noul nume trebuie să fie diferit de cel curent.")
```

```
if st.button("Șterge coloana"):
```

```
st.session_state.df = st.session_state.df.drop(columns=[selected_col])
```

```
st.success(f'Coloana '{selected_col}' a fost ștersă!')
```

```
st.rerun()
```

```
if st.button("Afișează numărul de valori lipsă"):
```

```
missing_count = st.session_state.df[selected_col].isna().sum()
```

```
st.info(f'Coloana '{selected_col}' are {missing_count} valori lipsă.")
```

Inițial, DataFrame-ul standardizat este încărcat și stocat în sesiunea curentă pentru a păstra modificările pe durata interacțiunii. Utilizatorul poate vizualiza conținutul complet al DataFrame-ului și poate selecta o coloană din lista disponibilă pentru a efectua diverse operații asupra acesteia. Printre funcționalitățile disponibile se numără redenumirea coloanei, cu verificarea ca noul nume să fie diferit de cel existent, ștergerea coloanei din DataFrame și afișarea numărului de valori lipsă din acea coloană.

- **Prelucrări statistice, gruparea și agregarea datelor cu pandas**

```
agg_diabetes = df_standardized[df_standardized['Diabetes'] == 1].groupby('Gender')[['BMI',  
'Blood Pressure', 'Cholesterol']].agg(['mean', 'sum', 'min', 'max', 'std']).round(2)
```

```
bins = [0, 20, 30, 40, 100]
```

```
labels = ['<20', '20-30', '30-40', '40+']
```

```
df_standardized['Age Group'] = pd.cut(df_cleaned['Age'], bins=bins, labels=labels,  
right=False)
```

```
age_group_agg = df_standardized.groupby('Age Group')[['Heart Rate', 'Blood Pressure', 'Cholesterol']].agg(['mean', 'sum', 'count', 'std']).round(2)
```

Variabila `agg_diabetes` filtrează datele pentru pacienții care au diabet (valoarea 1 la coloana `Diabetes`), grupează datele după `gen` și calculează pentru coloanele `BMI`, `Blood Pressure` și `Cholesterol` statistici precum media, suma, minimul, maximul și deviația standard, toate rotunjite la două zecimale.

Pentru analiza pe grupe de vârstă, variabila `Age Group` este creată prin împărțirea valorilor din coloana `Age` în intervale prestabilite (<20, 20-30, 30-40, 40+) folosind funcția `pd.cut`.

Variabila `age_group_agg` calculează pentru fiecare grupă de vârstă media, suma, numărul de observații și deviația standard pentru variabilele `Heart Rate`, `Blood Pressure` și `Cholesterol`, rotunjind toate rezultatele la două zecimale.

Age	Heart Rate				Blood Pressure				Cholesterol			
	mean	sum	count	std	mean	sum	count	std	mean	sum	count	std
18	0	39.62	10383	0.99	0	-4.97	10383	0.99	0	47.5	10383	1
19	0	-43.97	10413	1	0.02	185.42	10413	1.01	-0.01	-54.94	10413	0.99
20	-0.01	-85	10566	1	-0.01	-117.05	10566	1.02	-0.01	-87.19	10566	1.01
21	-0.01	-159.87	10677	1	0.01	125.29	10677	1	0	-29.25	10677	1
22	-0.02	-196.72	10691	1.01	0	-48.11	10691	0.99	-0.01	-125.67	10691	1
23	0.02	207.34	10335	0.99	0	1.62	10335	0.99	0.01	98.26	10335	1
24	-0.02	-161.02	10600	1	0.01	75.59	10600	0.98	0.01	69.03	10600	1
25	-0.01	-53.73	10683	1	0.01	138.05	10683	1	0.01	150.75	10683	1
26	0.01	166.42	30486	1	0	-68.71	30486	1	0.01	236.53	30486	1

Număr de fumători și persoane cu diabet pe grupe de vârstă					
AgeGroup	total_people	num_smokers	smokers_ratio	num_diabetics	diabetics_ratio
10-17	0	0	None	0	None
18-25	84348	15226	18.05	7446	8.83
26-30	73238	12873	17.58	6678	9.12
31+	42414	7930	18.7	3890	9.17

- Pentru intervalul de vârstă 18–25, dintr-un total de 84.348 persoane, există 15.226 fumători ($\approx 18\%$) și 7.446 persoane cu diabet ($\approx 8.83\%$).

- În grupa 26–30, din 73.238 persoane, sunt 12.873 fumători ($\approx 17.58\%$) și 6.678 diabetici ($\approx 9.12\%$).
- Pentru grupa 31+, din 42.414 persoane, sunt 7.930 fumători ($\approx 18.7\%$) și 3.890 diabetici ($\approx 9.17\%$).

Prelucrări statistice, gruparea și agregarea datelor în pachetul pandas

Agregare pe gen și fumat

	Gender	Smokir	BMI					Heart Rate					Blood Pressure					Cholesterol	
			mean	sum	min	max	std	mean	sum	min	max	std	mean	sum	min	max	std	mean	sum
0	0	0	23.35	1725660.61	10.13	44.31	6.67	79.55	5879000.67	60	99	10.91	114.65	8472528.92	90	139	13.65	184.6	1364217
1	0	1	23.33	375494.38	10.14	44.05	6.68	79.57	1280713.17	60	99	10.97	114.57	1844052.01	90	139	13.75	184.04	296213
2	1	0	23.34	2102261.52	10.07	44.36	6.68	79.47	7157657.26	60	99	10.98	114.48	10311362.01	90	139	13.66	184.5	1661832
3	1	1	23.29	464357.35	10.1	44.08	6.65	79.43	1583382.23	60	99	10.96	114.56	2283663.72	90	139	13.64	184.34	367464

Agregare pe diabet și gen

Gender	Diabetes	BMI					Blood Pressure					Cholesterol				
		mean	sum	min	max	std	mean	sum	min	max	std	mean	sum	min	max	std
0	0	23.34	1913357.62	10.13	44.31	6.66	114.64	9398358.92	90	139	13.68	184.51	15126995.37	120	249	35.61
0	1	23.44	187797.38	10.22	44.19	6.8	114.61	918222.01	90	139	13.61	184.39	1477308.98	120	249	35.28
1	0	23.33	2333048.77	10.07	44.36	6.66	114.5	11450600.05	90	139	13.66	184.41	18441787.42	120	249	35.68
1	1	23.35	233570.11	10.19	44.08	6.75	114.42	1144425.69	90	139	13.65	185.08	1851180.44	120	249	35.63

La nivelul agregării pe gen și statut de fumător, se observă o omogenitate relativă a valorilor medii pentru indicele de masă corporală (BMI ~ 23.3), ritmul cardiac (~ 79.5 bpm) și tensiunea arterială (~ 114.5 mmHg), în timp ce colesterolul prezintă variații mai pronunțate între subgrupuri (184.04-184.6 mg/dL). Agregarea pe diabet și gen dezvăluie următorul pattern: pacienții diabetici de gen masculin (grupa 1-1) prezintă cea mai mare valoare medie a colesterolului (185.08 mg/dL) și o ușoară creștere a BMI-ului (23.35), sugerând o posibilă asociere între diabetul zaharat, genul masculin și profilul lipidic alterat. Aceste observații susțin necesitatea unei abordări personalizate în managementul factorilor de risc cardiovascular, cu accent deosebit pe monitorizarea profilului lipidic la pacienții diabetici de sex masculin.

Vizualizare outliers

def plot_outliers(df, columns):

fig, axes = plt.subplots(nrows=len(columns), figsize=(10, 5 * len(columns)))

```

if len(columns) == 1:

axes = [axes]

for ax, col in zip(axes, columns):

sns.boxplot(x=df[col], ax=ax)

ax.set_title(f"Boxplot pentru {col}")

st.pyplot(fig)

```

Funcția `plot_outliers` creează grafice boxplot pentru fiecare coloană numerică din lista `columns` a DataFrame-ului `df`. Scopul este să identificăm și să vizualizăm eventualele valori anormale (outliers) din date, pentru a înțelege mai bine distribuția și variația fiecărei variabile analizate.

- **Reprezentări grafice**

Histogramă

```

if option == "Histogramă":

st.write("### Histogramă pentru date standardizate")

numeric_columns = df_standardized.select_dtypes(include=[np.number]).columns.tolist()

selected_column = st.selectbox("Alege o coloană numerică:", numeric_columns)

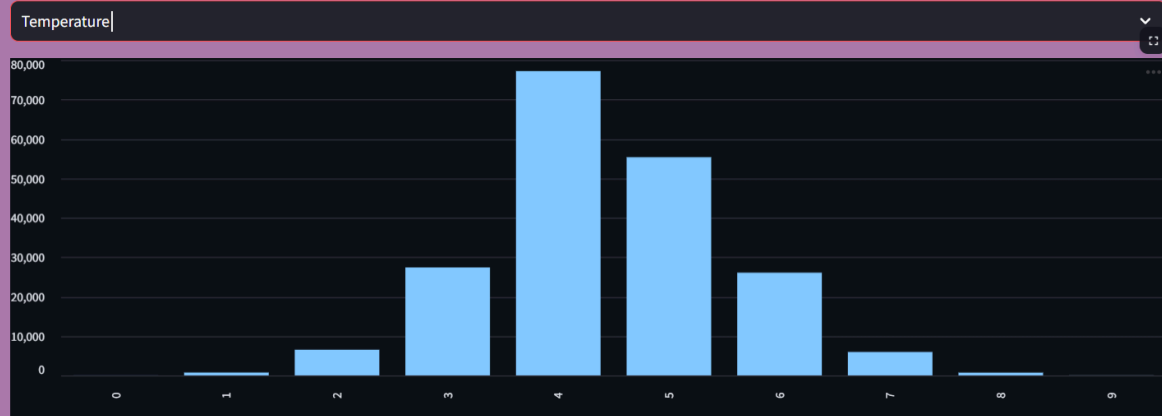
hist_values = np.histogram(df_standardized[selected_column].dropna(), bins=10)[0]

st.bar_chart(hist_values)

```

Histogramă pentru date standardizate

Alege o coloană numerică:



- **Clusterizare**

```
from sklearn.cluster import KMeans
```

```
from sklearn.decomposition import PCA
```

```
k = st.slider("Alege numărul de clustere", 2, 10, 3)
```

```
pca = PCA(n_components=2)
```

```
data_for_cluster = df_standardized[numeric_cols_to_scale]
```

```
reduced_data = pca.fit_transform(data_for_cluster)
```

```
kmeans = KMeans(n_clusters=k, random_state=0).fit(reduced_data)
```

```
cluster_labels = kmeans.labels_
```

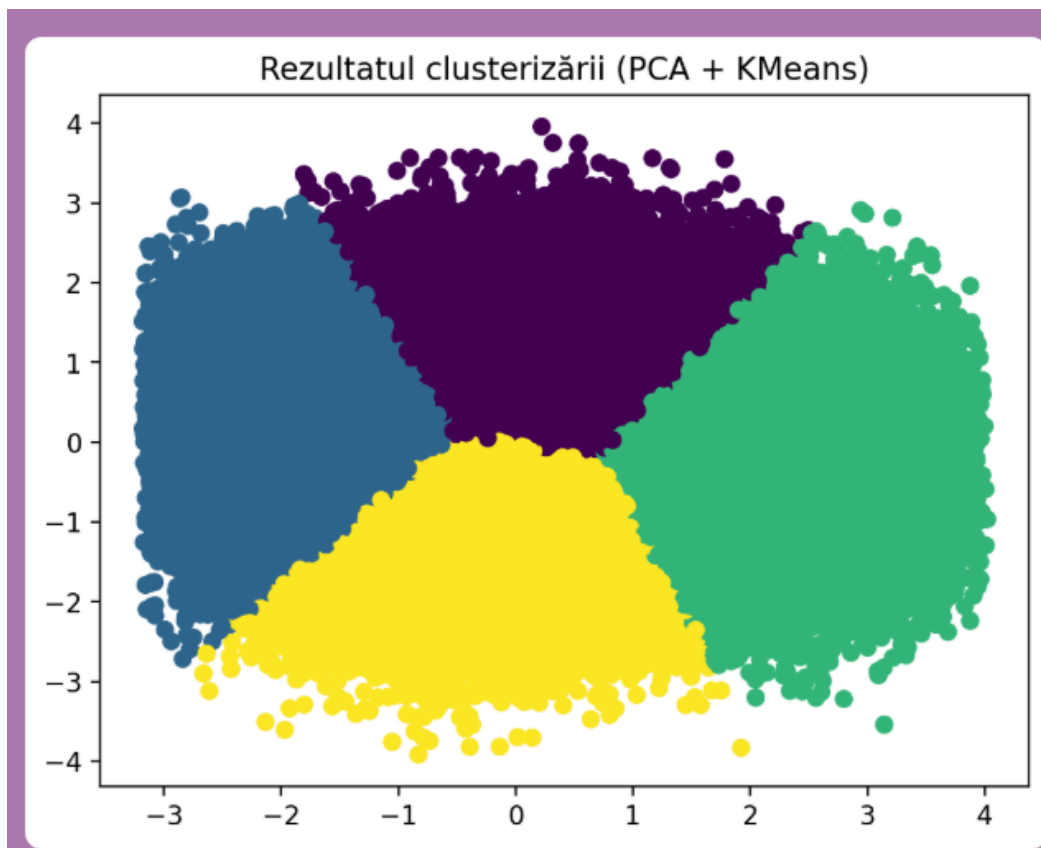
```
fig, ax = plt.subplots()
```

```
scatter = ax.scatter(reduced_data[:, 0], reduced_data[:, 1], c=cluster_labels, cmap="viridis")
```

```
ax.set_title("Rezultatul clusterizării (PCA + KMeans)")
```

```
st.pyplot(fig)
```

```
st.write(kmeans.cluster_centers_)
```



Se realizează o analiză de clusterizare KMeans aplicată asupra datelor standardizate, cu o vizualizare în două dimensiuni folosind PCA (Analiza Componentelor Principale). Mai întâi, utilizatorul poate selecta interactiv numărul de clustere dorite (între 2 și 10) prin intermediul unui slider în Streamlit. Datele numerice sunt reduse dimensional la două componente principale cu ajutorul PCA, pentru a putea fi reprezentate grafic în plan. Apoi, algoritmul KMeans este antrenat pe aceste date reduse, grupând observațiile în k clustere distincte. Rezultatul clusterizării este afișat într-un grafic scatter, unde fiecare punct reprezintă o observație colorată în funcție de clusterul căruia îi aparține. De asemenea, sunt afișate și coordonatele centroidelor clusterelor în spațiul redus PCA, oferind informații despre poziția medie a fiecărui grup. Această metodă combină reducerea dimensională pentru vizualizare clară și clusterizarea efectivă pentru identificarea grupărilor în date.

- **Regresie logistică**

```
X = df_cleaned[['Age', 'BMI', 'Blood Pressure', 'Cholesterol', 'Heart Rate']]

y = df_cleaned['Smoking']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

model = LogisticRegression(class_weight='balanced')

model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)

st.write("Acuratețea modelului:")

st.write(round(accuracy_score(y_test, y_pred), 2))

st.write("Matricea de confuzie:")

conf_matrix = pd.DataFrame(confusion_matrix(y_test, y_pred),

index=['Non-smoker (0)', 'Smoker (1)'],

columns=['Predicted 0', 'Predicted 1'])

st.dataframe(conf_matrix)

fig, ax = plt.subplots()

sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="Blues", ax=ax)

ax.set_ylabel("Actual")

ax.set_xlabel("Predicted")

st.pyplot(fig)
```

```

from sklearn.metrics import roc_auc_score, roc_curve

# ROC AUC

y_proba = model.predict_proba(X_test_scaled)[: , 1] # scorul pentru clasa "1" (fumători)

roc_auc = roc_auc_score(y_test, y_proba)

fpr, tpr, thresholds = roc_curve(y_test, y_proba)

st.write("Scor ROC-AUC:")

st.write(round(roc_auc, 2))

fig_roc, ax_roc = plt.subplots()

ax_roc.plot(fpr, tpr, label=f"ROC curve (AUC = {roc_auc:.2f})")

ax_roc.plot([0, 1], [0, 1], linestyle="--", color="gray") # Diagonală random

ax_roc.set_xlabel("False Positive Rate")

ax_roc.set_ylabel("True Positive Rate")

ax_roc.set_title("ROC Curve")

ax_roc.legend(loc="lower right")

st.pyplot(fig_roc)

```

Regresie logistică pentru predicția fumatului

Acuratețea modelului:

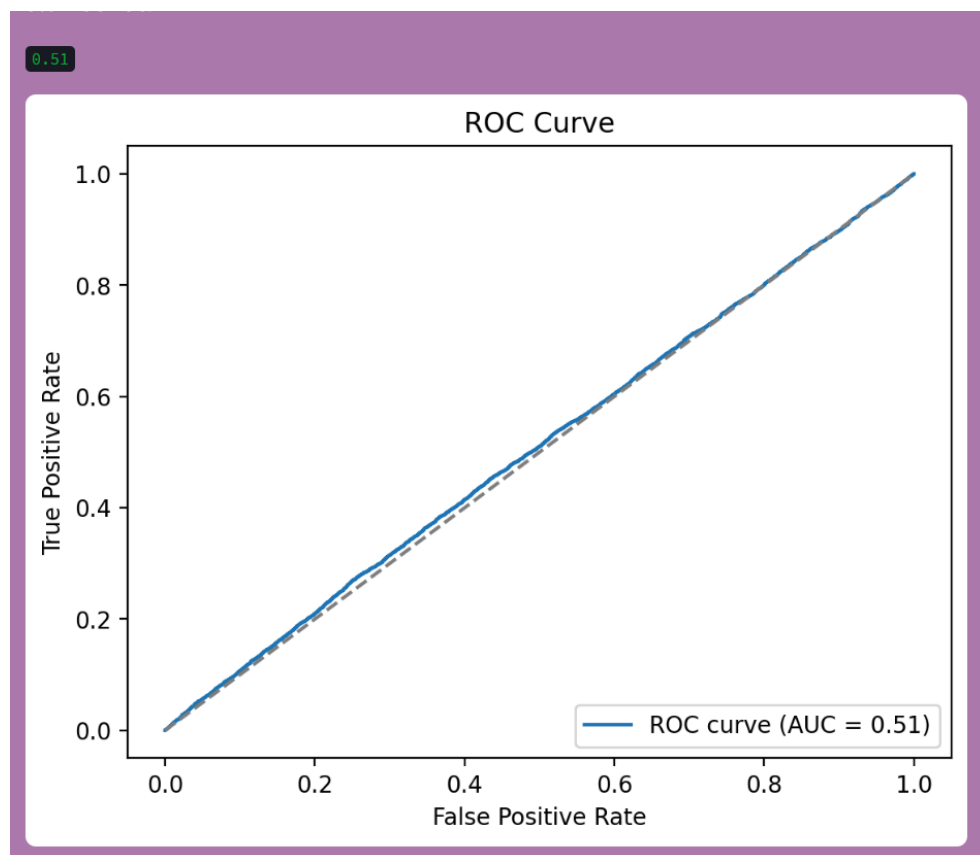
0.5

Matricea de confuzie:

	Predicted 0	Predicted 1
Non-smoker (0)	16276	16511
Smoker (1)	3503	3710

Pentru a analiza probabilitatea ca un individ să fie fumător, am utilizat un model de regresie logistică binară. Am dorit să clasificăm persoanele în două categorii: fumători (1) și nefumători (0), pe baza unor variabile explicative precum vârsta, indicele de masă corporală (BMI), tensiunea arterială, nivelul colesterolului și ritmul cardiac.

Datele au fost împărțite în două seturi, antrenare și testare, folosind un raport de 80%–20%, iar ulterior au fost standardizate cu ajutorul StandardScaler pentru a aduce toate variabilele pe aceeași scară. Pentru antrenarea modelului, am utilizat LogisticRegression în scikit-learn, cu opțiunea `class_weight='balanced'`, pentru a compensa dezechilibrul dintre clase (fumători vs. nefumători). Modelul a fost antrenat pe datele scalate și evaluat pe setul de testare, obținând o acuratețe de 0.5, ceea ce indică o performanță similară unei clasificări aleatorii. Matricea de confuzie a arătat că modelul clasifică mai bine nefumătorii, dar identifică slab fumătorii, având multe predicții greșite pentru această clasă. Deși acuratețea generală este 0.5, performanța pe clasa minoritară este slabă.



Graficul ROC arată o linie aproape diagonală, iar valoarea AUC=0.51 este foarte aproape de 0.5, ceea ce indică că modelul este aproape ca o alegere aleatorie.

- **Regresie Random Forest - Predicție fumători**

```
st.subheader("Random Forest")

st.write()

st.write("Acuratețe:")

st.write(round(accuracy_score(y_test, y_pred_rf), 2))

conf_matrix_rf = pd.DataFrame(confusion_matrix(y_test, y_pred_rf),
index=['Non-smoker (0)', 'Smoker (1)'],
columns=['Predicted 0', 'Predicted 1'])

st.write("Matricea de confuzie - Random Forest:")

st.dataframe(conf_matrix_rf)

st.write("Classification Report - Random Forest:")

report_rf = classification_report(y_test, y_pred_rf, output_dict=True)

st.dataframe(pd.DataFrame(report_rf).transpose().round(2))

# ROC-AUC

y_proba_rf = rf_model.predict_proba(X_test_scaled)[: , 1]

roc_auc_rf = roc_auc_score(y_test, y_proba_rf)

fpr_rf, tpr_rf, _ = roc_curve(y_test, y_proba_rf)

st.write("Scor ROC-AUC - Random Forest:")

st.write(round(roc_auc_rf, 2))
```

```
fig_rf, ax_rf = plt.subplots()

ax_rf.plot(fpr_rf, tpr_rf, label=f"ROC curve (AUC = {roc_auc_rf:.2f})")

ax_rf.plot([0, 1], [0, 1], linestyle="--", color="gray")

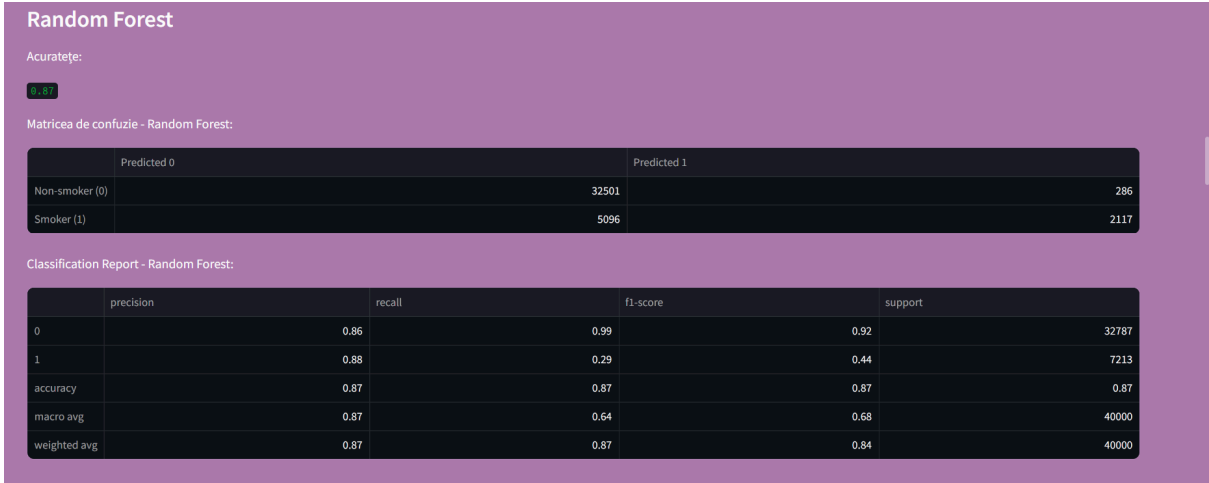
ax_rf.set_xlabel("False Positive Rate")

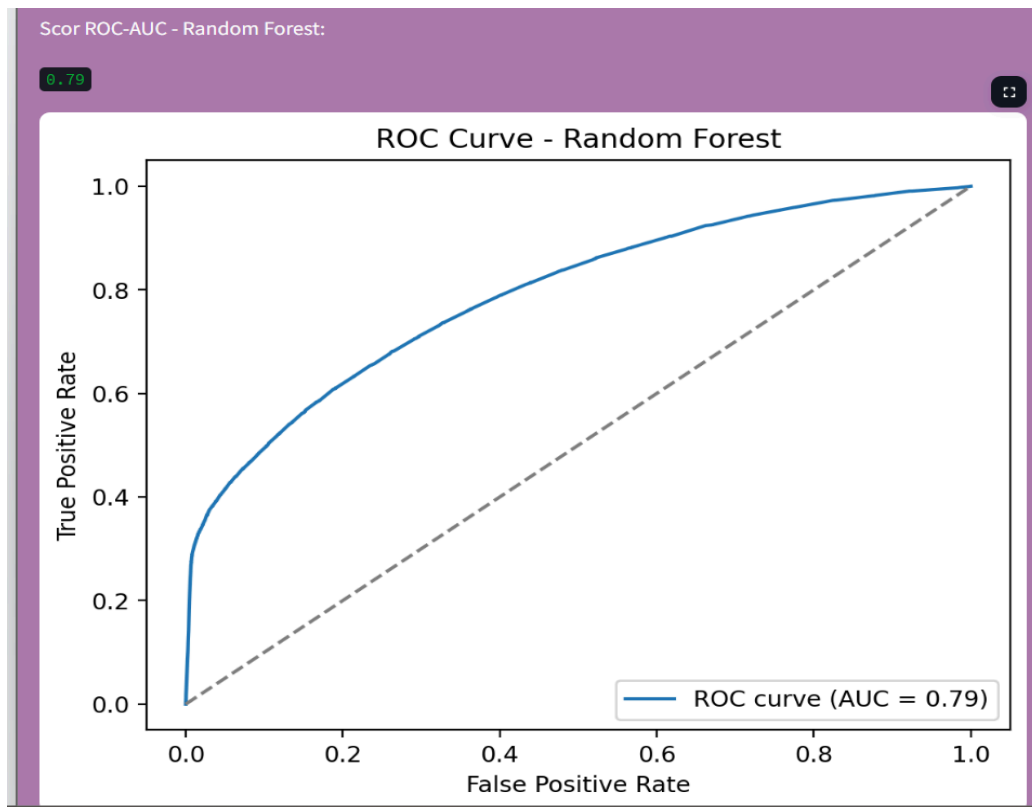
ax_rf.set_ylabel("True Positive Rate")

ax_rf.set_title("ROC Curve - Random Forest")

ax_rf.legend(loc="lower right")

st.pyplot(fig_rf)
```





Performanța generală a modelului este foarte bună, cu o **acuratețe de 87%** și un scor **ROC-AUC de 0.79**, indicând o capacitate solidă de discriminare între fumători și nefumători.

Matricea de confuzie relevă comportamentul modelului: din 32.787 de nefumători, modelul a identificat corect 32.501 (rata foarte mare de detectare a nefumătorilor), în timp ce din 7.213 fumători, doar 2.117 au fost identificați corect. Acest dezechilibru sugerează că modelul este conservator în predicția fumătorilor, preferând să clasifice incert cazurile ca nefumători.

Metricile de clasificare confirmă această tendință: pentru clasa 0 (nefumători), modelul atinge o precizie de 86% și un recall de 99%, demonstrând o capacitate excelentă de identificare a nefumătorilor cu foarte puține cazuri ratate. Pentru clasa 1 (fumători), precizia este de 88% (când prezice că cineva fumează, are dreptate în 88% din cazuri), dar recall-ul este doar 29%, ceea ce înseamnă că modelul ratează 71% din fumătorii reali.

Curba ROC cu AUC de 0.79 indică o performanță bună de separare între clase, fiind semnificativ mai bună decât clasificarea aleatoare (AUC = 0.5). Forma curbei sugerează că modelul poate fi optimizat prin ajustarea pragului de decizie pentru a îmbunătăți detectarea fumătorilor, dacă este necesar.

- **Regresie XGBoost - pentru predicția diabetului**

```
X = df_cleaned[['Age', 'BMI', 'Blood Pressure', 'Cholesterol', 'Heart Rate']]
```

```
y = df_cleaned['Diabetes']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
count_neg = sum(y_train == 0)
```

```
count_pos = sum(y_train == 1)
```

```
scale_pos_weight = count_neg / count_pos
```

```
xgb_model = xgb.XGBClassifier(
```

```
use_label_encoder=False,
```

```
eval_metric='logloss',
```

```
random_state=42,
```

```
scale_pos_weight=scale_pos_weight
```

```
)
```

```
xgb_model.fit(X_train_scaled, y_train)
```

```
y_pred_xgb = xgb_model.predict(X_test_scaled)
```

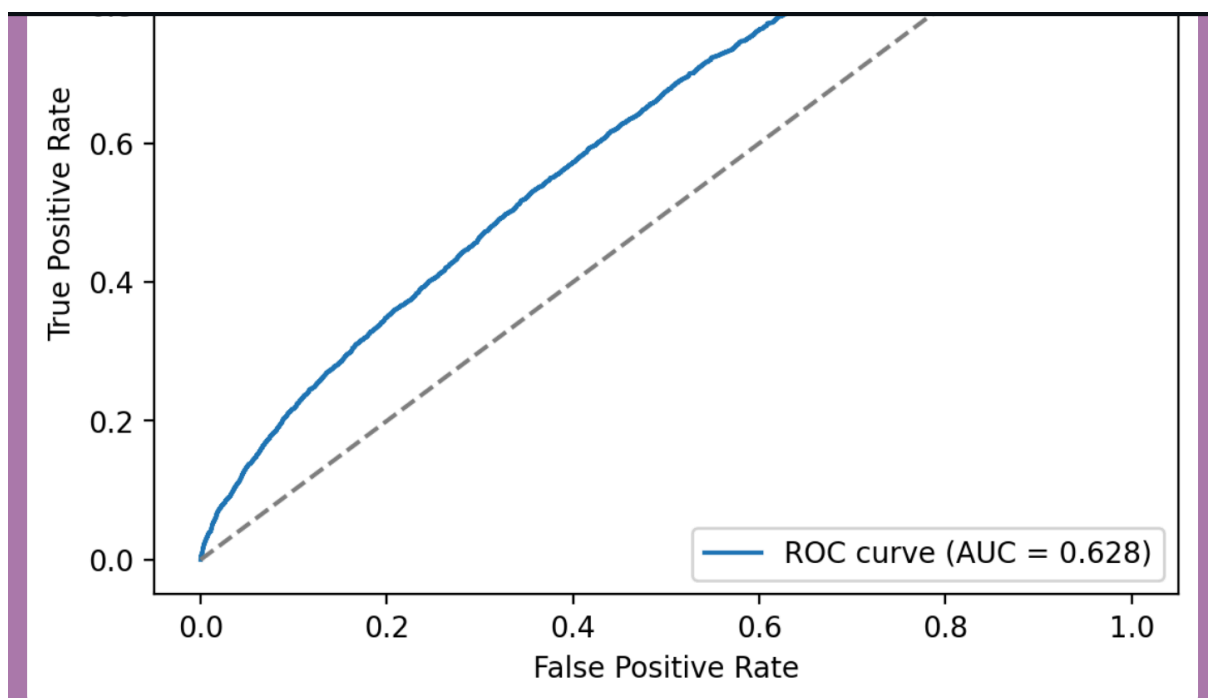
```
y_proba_xgb = xgb_model.predict_proba(X_test_scaled)[:, 1]
```

```
accuracy_xgb = accuracy_score(y_test, y_pred_xgb)
```

```
roc_auc_xgb = roc_auc_score(y_test, y_proba_xgb)
```

```
conf_matrix_xgb = pd.DataFrame(confusion_matrix(y_test, y_pred_xgb),  
index=['No Diabetes (0)', 'Diabetes (1)'],  
columns=['Predicted 0', 'Predicted 1'])  
  
st.subheader("XGBoost echilibrat pentru predicția diabetului")  
  
st.write("Acuratețe:")  
  
st.write(round(accuracy_xgb, 3))  
  
st.write("Matricea de confuzie:")  
  
st.dataframe(conf_matrix_xgb)  
  
st.write("Classification Report:")  
  
report_xgb = classification_report(y_test, y_pred_xgb, output_dict=True)  
  
st.dataframe(pd.DataFrame(report_xgb).transpose().round(2))  
  
st.write("ROC-AUC Score:")  
  
st.write(round(roc_auc_xgb, 3))  
  
fpr_xgb, tpr_xgb, _ = roc_curve(y_test, y_proba_xgb)  
  
fig_xgb, ax_xgb = plt.subplots()  
  
ax_xgb.plot(fpr_xgb, tpr_xgb, label=f"ROC curve (AUC = {roc_auc_xgb:.3f})")  
  
ax_xgb.plot([0, 1], [0, 1], linestyle="--", color="gray")  
  
ax_xgb.set_xlabel("False Positive Rate")  
  
ax_xgb.set_ylabel("True Positive Rate")  
  
ax_xgb.set_title("ROC Curve - XGBoost")  
  
ax_xgb.legend(loc="lower right")
```

st.pyplot(fig_xgb)



Modelul XGBoost utilizat pentru predicția diabetului are o acuratețe de 65.4%, însă performanțele sunt dezechilibrate între clase. Matricea de confuzie și raportul de clasificare arată că modelul identifică bine cazurile fără diabet (recall de 93% pentru clasa 0), dar prezintă dificultăți majore în detectarea cazurilor reale de diabet (recall de doar 13% pentru clasa 1). F1-score-ul pentru clasa 1 este scăzut (0.5), indicând o capacitate slabă de

generalizare pentru cazurile pozitive. AUC-ul ROC de 0.628 confirmă performanța modestă a modelului în separarea celor două clase, fiind aproape de linia de bază (0.5).

- **Regresie multiplă**
 - **Analiza regresiei multiple pentru BMI în funcție de înălțime și greutate**

```
X_multi = df_cleaned[['Height', 'Weight']]
```

```
y_multi = df_cleaned['BMI']
```

```
X_multi_with_const = sm.add_constant(X_multi)
```

```
model_multi = sm.OLS(y_multi, X_multi_with_const).fit()
```

```
st.text("Rezultatele modelului de regresie multiplă:")
```

```
st.text(model_multi.summary())
```

OLS Regression Results

=====

Dep. Variable:	BMI	R-squared:	0.792
Model:	OLS	Adj. R-squared:	0.792
Method:	Least Squares	F-statistic:	3.807e+05
Date:	Wed, 04 Jun 2025	Prob (F-statistic):	0.00
Time:	19:49:47	Log-Likelihood:	-5.0638e+05
No. Observations:	200000	AIC:	1.013e+06
Df Residuals:	199997	BIC:	1.013e+06
Df Model:	2		
Covariance Type:	nonrobust		

coef	std err	t	P> t	[0.025	0.975]
const	44.5821	0.092	485.469	0.000	44.402 44.762
Height	-0.2414	0.000	-486.192	0.000	-0.242 -0.240
Weight	0.3000	0.000	724.367	0.000	0.299 0.301
=====					
Omnibus:	21123.622	Durbin-Watson:	1.996		
Prob(Omnibus):	0.000	Jarque Bera (JB):	149718.865		
Skew:	0.253	Prob(JB):	0.00		
Kurtosis:	7.208	Cond. No.	2.55e+03		

Modelul de regresie liniară multiplă are ca scop estimarea valorii indicelui de masă corporală (BMI) în funcție de două variabile explicative: înălțimea (Height) și greutatea (Weight) individului. Ecuația regresiei obținute este:

$$\text{BMI} = 44.5821 - 0.2414 \times \text{Height} + 0.3000 \times \text{Weight}.$$

Interpretarea coeficientilor relevă că greutatea are un impact pozitiv asupra BMI: la o înălțime constantă, o creștere cu 1 kilogram a greutății determină, în medie, o creștere a BMI cu 0.3 unități. Pe de altă parte, înălțimea are un efect negativ asupra BMI: la o greutate constantă, o creștere cu 1 centimetru a înălțimii este asociată cu o scădere a BMI cu aproximativ 0.24 unități. Ambii coeficienți sunt semnificativi statistic, având valori p sub 0.001, ceea ce înseamnă că impactul lor asupra variabilei dependente este real, nu întâmplător.

Calitatea modelului este una ridicată, indicată de coeficientul R-squared de 0.792. Acesta arată că aproximativ 79.2% din variația valorilor BMI este explicată de modelul construit pe baza greutății și înălțimii. De asemenea, valoarea ajustată a lui R-squared este

identică (0.792), semnalând că ambele variabile sunt relevante pentru model. Testul F este foarte semnificativ ($p < 0.001$), confirmând că modelul, în ansamblu, este valid din punct de vedere statistic.

Valoarea statisticii Durbin-Watson este de 1.996, aproape de valoarea ideală de 2, ceea ce sugerează că nu există autocorelație a erorilor. Deși testul Jarque-Bera indică o anumită abatere de la normalitatea reziduurilor, acest aspect este acceptabil având în vedere dimensiunea foarte mare a eșantionului. Condiționarea numerică (Cond. No ≈ 2550) nu ridică probleme de coliniaritate severă.

Analiza regresiei multiple pentru atac de cord în funcție de temperatură și greutate și vârstă

```
st.write("Atac de cord - tinta si temperatura, greutate, varsta")
```

```
X_multi = df_cleaned[['Temperature', 'Weight', 'Age']]
```

```
y_multi = df_cleaned['Heart Rate']
```

```
X_multi_with_const = sm.add_constant(X_multi)
```

```
model_multi = sm.OLS(y_multi, X_multi_with_const).fit()
```

```
st.text("Rezultatele modelului de regresie multiplă:")
```

```
st.text(model_multi.summary())
```

OLS Regression Results

```
=====
Dep. Variable:          Heart Rate  R-squared:                0.000
Model:                  OLS  Adj. R-squared:              0.000
Method:                 Least Squares  F-statistic:             2.136
Date:                  Wed, 04 Jun 2025  Prob (F-statistic):      0.0934
Time:                  19:49:48  Log-Likelihood:          -7.6242e+05
No. Observations:      200000  AIC:                      1.525e+06
Df Residuals:          199996  BIC:                      1.525e+06
```

Df Model: 3
Covariance Type: nonrobust

	coef	std err	t	P> t	[0.025	0.975]
const	87.6069	5.086	17.225	0.000	77.638	97.575
Temperature	-0.0848	0.052	-1.645	0.100	-0.186	0.016
Weight	-5.09e-05	0.001	-0.034	0.973	-0.003	0.003
Age	0.0102	0.005	1.928	0.054	-0.000	0.021
Omnibus:	45456.018	Durbin-Watson:	1.991			
Prob(Omnibus):	0.000	Jarque Bera (JB):	8297.538			
Skew:	-0.001	Prob(JB):	0.00			
Kurtosis:	2.002	Cond. No.	2.58e+04			

Modelul de regresie liniară multiplă are ca scop explicarea valorii ritmului cardiac (Heart Rate) în funcție de trei variabile explicative: temperatura corporală (Temperature), greutatea (Weight) și vârsta (Age) individului. Ecuația regresiei rezultate este:

$$\text{Heart Rate} = 87.6069 - 0.0848 \times \text{Temperature} - 0.0000509 \times \text{Weight} + 0.0102 \times \text{Age}.$$

Interpretarea coeficientilor indică faptul că nicio variabilă nu are un impact semnificativ asupra ritmului cardiac, din punct de vedere statistic. Coeficientul asociat temperaturii (-0.0848) sugerează că o creștere cu 1 grad Fahrenheit ar putea duce, în medie, la o scădere a ritmului cardiac cu aproximativ 0.08 bătăi pe minut, însă această relație nu este semnificativă ($p = 0.100$). Greutatea are un coeficient practic nul (-0.0000509), ceea ce indică lipsa oricărui efect real asupra ritmului cardiac, fapt susținut de valoarea p foarte mare ($p = 0.973$). Vârsta are un efect ușor pozitiv (coeficient = 0.0102), dar și acesta este la limita semnificației statistice ($p = 0.054$), deci nu poate fi considerat relevant din punct de vedere riguros.

Calitatea generală a modelului este foarte slabă. Coeficientul R^2 este practic zero (0.000), ceea ce înseamnă că modelul nu explică deloc variația ritmului cardiac în funcție de variabilele incluse. De asemenea, valoarea ajustată a lui R^2 confirmă acest aspect, fiind tot zero. Testul F are o valoare scăzută (2.136) și un p -value de 0.0934, indicând că modelul, în

ansamblu, nu este semnificativ statistic. Cu alte cuvinte, nu există dovezi solide că variabilele introduse în model ar avea un impact real asupra variabilei dependente.

Valoarea Durbin-Watson este de 1.991, foarte apropiată de 2, ceea ce sugerează că nu există autocorelare a erorilor. Rezultatele testului Jarque-Bera arată că reziduurile nu respectă complet normalitatea, dar acest aspect este puțin relevant în contextul unui model cu putere explicativă aproape nulă. Valoarea Cond. No (≈ 25.800) nu indică coliniaritate severă.

II) SAS

Pentru programarea SAS am folosit același set de date ca mai sus la Python.

- **Importul setului de date**

```
proc import datafile="/home/u64210696/medical_students_dataset_cleaned.csv"
  out=muncadata
  dbms=csv
  replace;
  getnames=yes;
run;
```

Setul de date SAS muncadata a fost creat prin importul fișierului CSV extern folosind procedura PROC IMPORT. Astfel am transformat datele din formatul extern într-un format intern SAS, pregătit pentru prelucrare.

CODE	LOG	RESULTS	OUTPUT DATA
Table: WORK.MUNCADATA	View: Column names	Filter: (none)	
Columns	Total rows: 200000 Total columns: 12		
☒ Select all ☒ Age ☒ Gender ☒ Height ☒ Weight ☒ Blood Type ☒ BMI ☒ Temperature ☒ Heart Rate ☒ Blood Pressure	Rows 1-100		
Property	Value		
Label			
Name			
Length			
Type			
Format			
Informat			
Age	Gender	Height	Weight Blood Type
1	18 Female	161.7779242	72.35494708 O
2	26 Male	152.0691568	47.6309415 B
3	32 Female	182.5376635	55.74108279 A
4	30 Male	182.1128672	63.33220706 B
5	23 Female	174.94710267	46.23417253 O
6	32 Male	151.4912942	68.64780494 B
7	21 Male	172.9497041	48.10274387 AB
8	28 Male	186.4894021	52.38975193 AB
9	21 Male	155.0396784	42.95870258 B
10	32 Male	170.8363146	50.78324973 B
11	28 Female	152.9739277	73.57281402 B
12	34 Female	182.4163021	76.37105033 AB
13	29 Male	168.845992	48.53356461 AB
14	34 Male	174.94710267	60.8822282 B
15	33 Male	184.7189883	93.66694446 B
16	21 Male	162.8197182	96.38569806 B
17	24 Male	162.7270126	86.22872653 B

- **Formate definite de utilizator**

proc format;

value diabet_fmt

1 = 'Diabetic'

0 = 'Non-diabetic'

other = 'Necunoscut';

value \$fum

'Yes' = 'Fumator'

'No' = 'Nefumator'

other = 'Necunoscut';

value \$gen

'Male' = 'Barbat'

'Female' = 'Femeie'

other = 'Necunoscut';

value \$grups

'A' = 'A'

```

'B' = 'B'
'AB' = 'AB'
'O' = 'O'
other = 'Altul';

run;

proc print data=muncadata (obs=10);
    format Diab_bin diabet_fmt. Smoking $fum. Gender $gen. Blood_Type $grups.;
run;

```

Obs	Age	Gender	Height	Weight	Blood Type	BMI	Temperature	Heart Rate	Blood Pressure	Cholesterol	Diabetes	Smoking	Diab_bin
1	18	Femeie	161.7779242	72.35494708	O	27.64583507	98.600947877	95	109	203	0	0	Non-diabetic
2	26	Barbat	152.0691568	47.6309415	B	23.33886936	98.71497675	93	104	163	0	0	Non-diabetic
3	32	Femeie	182.5376635	55.74108279	A	16.72901714	98.26029302	76	130	216	1	0	Diabetic
4	30	Barbat	182.1128672	63.33220706	B	19.09604191	98.83960472	99	112	141	0	1	Non-diabetic
5	23	Femeie	174.94710267	46.23417253	O	23.33886936	98.48000771	95	114.55803333	231	0	0	Non-diabetic
6	32	Barbat	151.4912942	68.64780494	B	29.91240299	99.66837273	70	128	183	0	1	Non-diabetic
7	21	Barbat	172.9497041	48.10274387	AB	16.08163509	97.71546872	66	134	247	0	0	Non-diabetic
8	28	Barbat	186.4894021	52.38975193	AB	15.06392098	98.22778787	85	123	128	0	0	Non-diabetic
9	21	Barbat	155.0396784	42.95870258	B	23.33886936	98.80805323	79.503766667	111	243	0	0	Non-diabetic
10	32	Barbat	170.8363146	50.78324973	B	17.40043481	98.57016848	61	94	166	0	0	Non-diabetic

- **Crearea de subseturi de date**

```

data subset_diabet;
    set muncadata;
    if Diab_bin = 1;
run;

data subset_nondiabet;
    set muncadata;
    if Diab_bin = 0;
run;

data subset_fumatori;
    set muncadata;
    if Smoke_bin = 1;
run;

```

```

data subset_femei_diabet;
  set muncadata;
  if Gender_bin = 0 and Diab_bin = 1;
run;

```

Total rows: 18014 Total columns: 15

Rows 1-100

Smoking	Diab_bin	Gender_bin	Smoke_bin
0	1	0	0
0	1	1	0
1	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1
1	1	0	1
0	1	1	0
0	1	0	0
0	1	1	0
1	1	0	1
0	1	1	0
0	1	1	0
0	1	0	0
0	1	1	0
0	1	0	0
0	1	0	0
0	1	0	0
0	1	0	0
0	1	0	0
0	1	0	0
0	1	0	0

- **Utilizarea de funcții SAS**

```
data muncadata;
```

```
  set muncadata;
```

```
  if Age > 0 then ColesterolPeVarsta = Cholesterol / Age;
```

```
  else ColesterolPeVarsta = .;
```

```
  if Height > 0 then RaportGreutateInaltime = Weight / Height;
```

```
  else RaportGreutateInaltime = .;
```

```
  if BMI ne . then BMI_rotunjit = round(BMI, 0.1);
```

```
else BMI_rotunjit = .;
```

```
if Gender ne " then Gen_initial = substr(Gender, 1, 1);
```

```
else Gen_initial = ";
```

```
if Temperature > 99 then Febra = 'DA';
```

```
else if Temperature ne . then Febra = 'NU';
```

```
else Febra = 'NECUNOSCU';
```

```
run;
```

În această secțiune au fost aplicate funcții SAS pentru a calcula variabile suplimentare relevante clinic și analitic. S-a utilizat funcția round pentru a aproxima indicele BMI la o zecimală, în scopul standardizării valorilor. Funcția substr a fost folosită pentru a extrage prima literă a genului (masculin/feminin), utilă în analize rapide de segmentare. Funcția division (/) a fost aplicată condiționat pentru a calcula raporturi precum colesterol raportat la vârstă (Cholesterol / Age) sau greutate raportată la înălțime (Weight / Height), evitând împărțirea la zero.

În plus, s-a introdus o variabilă binară (Febra) derivată din valoarea temperaturii corporale, utilizând o structură condițională if-else pentru a marca depășirea pragului clinic de 99°F.

ColesterolPeVarsta	RaportGreutateInaltime	BMI_rotunjit	Gen_initial	Febra
11.277777778	0.4472485813	27.6	F	NU
6.2692307692	0.3132189492	23.3	M	NU
6.75	0.3053675703	16.7	F	NU
4.7	0.3477634943	19.1	M	NU
10.043478261	0.2642751542	23.3	F	NU
5.71875	0.4531468643	29.9	M	DA
11.761904762	0.2781314031	16.1	M	NU
4.5714285714	0.2809261617	15.1	M	NU
11.571428571	0.2770819897	23.3	M	NU
5.1875	0.2972626157	17.4	M	NU
8.6071428571	0.4809500228	31.4	F	NU
7.2647058824	0.4186635155	23	F	NU
8.4827586207	0.2874427994	23.3	M	NU
7.1470588235	0.3480036381	22.5	M	NU
5.4545454545	0.5070780504	27.5	M	NU
6.1904761905	0.5919780425	23.3	M	NU
8.2083333333	0.5298980492	32.6	M	NU

- **Combinarea seturilor de date prin proceduri specifice SAS și SQL**

```
data pacienti_baza;
```

```
    set muncadata (keep=Age Gender BMI Diab_bin);
```

```
run;
```

```
data variabile_vitale;
```

```
    set muncadata (keep=Heart_Rate Blood_Pressure Cholesterol Diab_bin);
```

```
run;
```

```
proc sort data=pacienti_baza;
```

```
    by Diab_bin;
```

```
run;
```

```
proc sort data=variabile_vitale;
```

```
    by Diab_bin;
```

```
run;
```

```
data date_completate;
```

```
merge pacienti_baza variabile_vitale;
by Diab_bin;
run;
```

```
proc sql;
create table date_sql as
select a.*, b.Temperature, b.Smoke_bin
from muncadata a
inner join muncadata b
on a.Diab_bin = b.Diab_bin;
quit;
```

Au fost demonstrate două metode de combinare a seturilor de date în SAS. Prima abordare utilizează metode clasice, prin sortarea datelor după cheia comună (Diab_bin) și aplicarea instrucțiunii merge.

A doua abordare folosește proc sql pentru realizarea unui inner join între două instanțe ale aceluiași set de date (muncadata), în funcție de variabila Diab Bin.

Total rows: 200000 Total columns: 4

	Age	Gender	BMI	Diab_bin
1	18	Female	27.64583507	0
2	26	Male	23.33886936	0
3	30	Male	19.09604191	0
4	23	Female	23.33886936	0
5	32	Male	29.91240299	0
5	21	Male	16.08163509	0
7	28	Male	15.06392098	0
3	21	Male	23.33886936	0
7	32	Male	17.40043481	0
10	28	Female	31.43999996	0
11	34	Female	22.95099236	0
12	29	Male	23.33886936	0
13	21	Male	23.33886936	0
14	24	Male	32.56361934	0
15	27	Male	18.01463828	0
16	31	Female	18.57272337	0
17	31	Male	18.02220703	0

- **Utilizarea de masive**

```
data masive_vitale;
set muncadata;
array vitale[4] Temperature Heart_Rate Blood_Pressure Cholesterol;
array marcaje[4] $10 _temporary_ ('Temp' 'HR' 'BP' 'Col');

do i = 1 to 4;
    if vitale[i] ne . then do;
        Indicator = vitale[i];
        Tip = marcaje[i];
        output;
    end;
end;

keep Diab_bin Indicator Tip;
run;
```

În această secțiune a fost utilizată structura array pentru a grupa patru variabile vitale relevante clinic: temperatura corporală, ritmul cardiac, tensiunea arterială și colesterolul. Fiecare valoare validă este transformată într-o observație distinctă, cu ajutorul unei bucle do, însoțită de un vector temporary care etichetează tipul indicatorului (Temp, HR, BP, Col).

Total rows: 400000 Total columns: 3

	Diab_bin	Indicator	Tip
1	0	98.600947877	Temp
2	0	203	Col
3	0	98.71497675	Temp
4	0	163	Col
5	1	98.26029302	Temp
6	1	216	Col
7	0	98.83960472	Temp
8	0	141	Col
9	0	98.48000771	Temp
10	0	231	Col
11	0	99.66837273	Temp
12	0	183	Col
13	0	97.71546872	Temp
14	0	247	Col
15	0	98.22778787	Temp
16	0	128	Col
17	0	98.80805323	Temp

- **Proceduri pentru raportare și proceduri statistice + generare de grafice**

```
proc freq data=muncadata;
```

```
tables Diabetes Smoking Gender / missing;
```

```
run;
```

```
proc means data=muncadata n mean min max std;
```

```
var Age BMI Cholesterol Temperature Heart_Rate Blood_Pressure;
```

```
run;
```

```
proc univariate data=muncadata;
```

```
var BMI;
```

```
histogram BMI;
```

```
inset mean std min max / position=ne;
```

```
run;
```

```
proc sgplot data=muncadata;
```

```
vbar Diabetes / group=Smoking groupdisplay=cluster;
```

```
run;
```

```
proc sgplot data=muncadata;
```

```
histogram Cholesterol / binwidth=10;
```

```
density Cholesterol;
```

```
run;
```

Această secțiune include raportarea descriptivă și analiză statistică de bază, în conformitate cu cerințele standard din seminare. PROC FREQ oferă frecvențele categoriilor pentru variabile cheie precum Diabetes, Smoking și Gender, inclusiv valorile lipsă. PROC MEANS furnizează statistici descriptive pentru variabile continue, iar PROC UNIVARIATE detaliază distribuția indicelui BMI, cu histogramă și valori numerice esențiale afișate în grafic. PROC SGPLOT este utilizat pentru a genera diagrame cu bare (compararea Diabetes în funcție de Smoking) și o histogramă cu densitate pentru colesterol.

The FREQ Procedure

Diabetes	Frequency	Percent	Cumulative Frequency	Cumulative Percent
0	181986	90.99	181986	90.99
1	18014	9.01	200000	100.00

Smoking	Frequency	Percent	Cumulative Frequency	Cumulative Percent
0	163971	81.99	163971	81.99
1	36029	18.01	200000	100.00

Gender	Frequency	Percent	Cumulative Frequency	Cumulative Percent
Female	89995	45.00	89995	45.00
Male	110005	55.00	200000	100.00

Distribuția variabilelor arată că 9,01% dintre persoanele din eșantion prezintă diabet, în timp ce 90,99% nu suferă de această afecțiune, ceea ce indică un dezechilibru de clasă important. Aproximativ 18% dintre respondenți sunt fumători, în timp ce 82% nu fumează. În ceea ce privește genul, 55% dintre persoane sunt de sex masculin, iar 45% feminin, ceea ce sugerează o ușoară suprasolicitare pe genul masculin.

The UNIVARIATE Procedure
Variable: BMI

Moments			
N	200000	Sum Weights	200000
Mean	23.3388694	Sum Observations	4667773.87
Std Deviation	6.67261306	Variance	44.5237651
Skewness	0.46206203	Kurtosis	-0.1385878
Uncorrected SS	117845273	Corrected SS	8904708.49
Coeff Variation	28.5901299	Std Error Mean	0.01492042

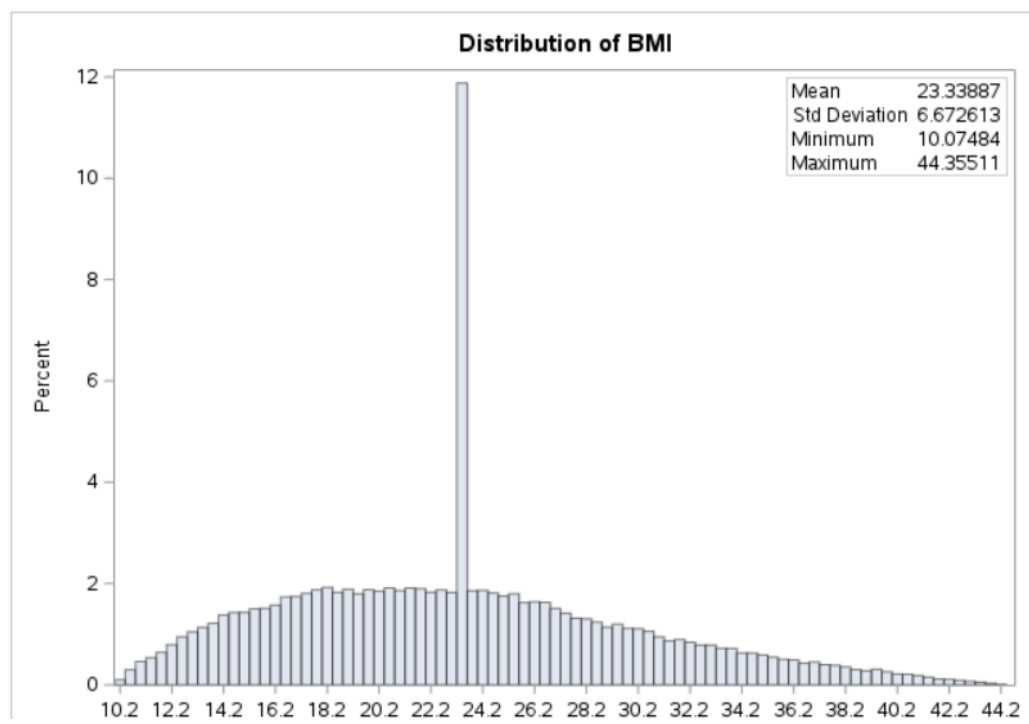
Basic Statistical Measures			
Location		Variability	
Mean	23.33887	Std Deviation	6.67261
Median	23.33887	Variance	44.52377
Mode	23.33887	Range	34.28028
		Interquartile Range	8.87271

Tests for Location: Mu0=0				
Test	Statistic		p Value	
Student's t	t	1564.224	Pr > t 	<.0001
Sign	M	100000	Pr >= M 	<.0001
Signed Rank	S	1E10	Pr >= S 	<.0001

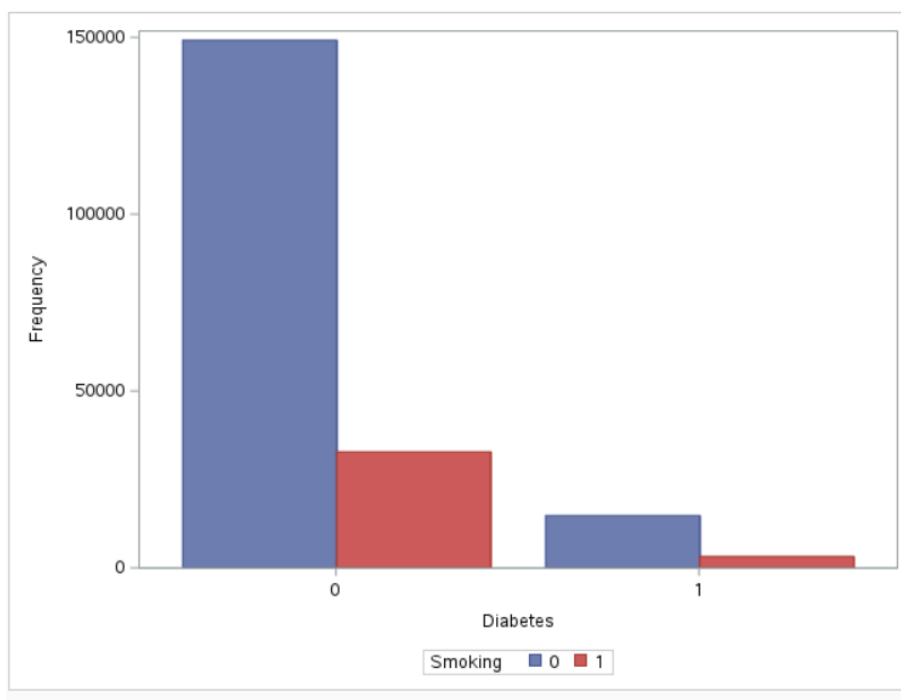
Quantiles (Definition 5)	
Level	Quantile
100% Max	44.3551
99%	40.4223
95%	35.9076
90%	32.8008
75% Q3	27.2555
50% Median	23.3389
25% Q1	18.3828
10%	14.8076
5%	13.2685
1%	11.3149
0% Min	10.0748

Extreme Observations			
Lowest		Highest	
Value	Obs	Value	Obs
10.0748	162661	44.2880	9455
10.0748	62661	44.2880	109455
10.0814	118386	44.3141	19313
10.0814	18386	44.3141	119313
10.0901	188038	44.3551	100394

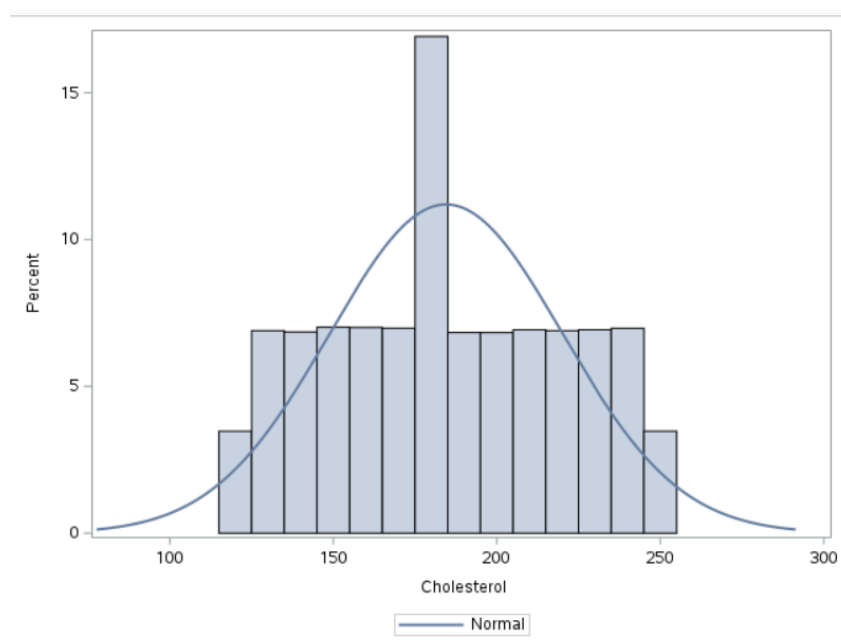
Distribuția indicelui BMI în setul analizat are o medie de 23,34 și o deviație standard de aproximativ 6,67, ceea ce sugerează o dispersie moderată în jurul valorii medii. Distribuția este ușor asimetrică spre dreapta (skewness = 0,46), iar valoarea kurtosis apropiată de zero indică o formă apropiată de distribuția normală. Media, mediana și moda coincid (23,34), ceea ce confirmă simetria centrală. Valorile extreme variază de la 10,07 la 44,36, iar 50% din populație are un BMI între 18,38 (Q1) și 27,26 (Q3). Rezultatele sugerează că majoritatea indivizilor au un indice BMI în intervalul normal, cu prezența câtorva valori în zona obezității.



Histograma indicelui BMI arată o distribuție aproape simetrică, centrată în jurul valorii de 23,3, cu o acumulare bruscă și neobișnuit de mare într-o singură clasă (probabil cauzată de valori duplicate sau rotunjiri în date). Majoritatea valorilor se situează între 18 și 30, iar distribuția scade treptat spre extremitatea dreaptă, până la un maxim de aproximativ 44.



Graficul arată distribuția cazurilor de diabet în funcție de statutul de fumător. Observăm că numărul persoanelor fără diabet este semnificativ mai mare, atât în rândul nefumătorilor, cât și al fumătorilor. Totuși, proporțional, există mai mulți fumători în rândul celor diagnosticați cu diabet decât în rândul celor sănătoși. Acest lucru sugerează o posibilă asociere între fumat și prezența diabetului.



Histograma colesterolului arată o distribuție relativ uniformă, cu un vârf central clar. Curba de densitate suprapusă indică o tentativă de ajustare la o distribuție normală, însă distribuția reală este mai plată și prezintă o frecvență ridicată la o valoare fixă din centru. Deși valorile sunt distribuite în intervalul 100–260 mg/dL, acumularea artificială din centru sugerează că această variabilă ar putea influența modelarea și ar trebui evaluată pentru tratamentul valorilor repetate.

- **SAS ML**

```
data muncadata_model;
```

```
set muncadata;
```

```
if Diabetes = 'Yes' then Target = 1;
```

```
else if Diabetes = 'No' then Target = 0;
```

```
else Target = .;
```

```
if Gender = 'Male' then Gender_bin = 1;
```

```
else if Gender = 'Female' then Gender_bin = 0;
```

```
else Gender_bin = .;
```

```
if Smoking = 'Yes' then Smoke_bin = 1;
```

```
else if Smoking = 'No' then Smoke_bin = 0;
```

```
else Smoke_bin = .;
```

```
if BMI ne . then BMI_grup = (BMI >= 25);
```

```
run;
```

```
proc stdize data=muncadata_model out=muncadata_scaled method=range;
```

```
var Age BMI Cholesterol Heart_Rate Blood_Pressure Temperature;
```

```
run;
```

```
proc surveyselect data=muncadata_scaled out=split_data seed=2025
```

```
  samprate=0.7 outall method=srs;
```

```
run;
```

```

data train test;
    set split data;
    if selected then output train;
    else output test;
run;

%let vars = Age BMI Cholesterol Heart_Rate Blood_Pressure Temperature Gender_bin
Smoke_bin BMI_grup;

proc logistic data=train descending outmodel=log_model;
    model Target = &vars;
    score data=test out=pred_logit outroc=roc_logit;
run;

data pred_logit;
    set pred_logit;
    pred_class = (P_1 > 0.5);
run;

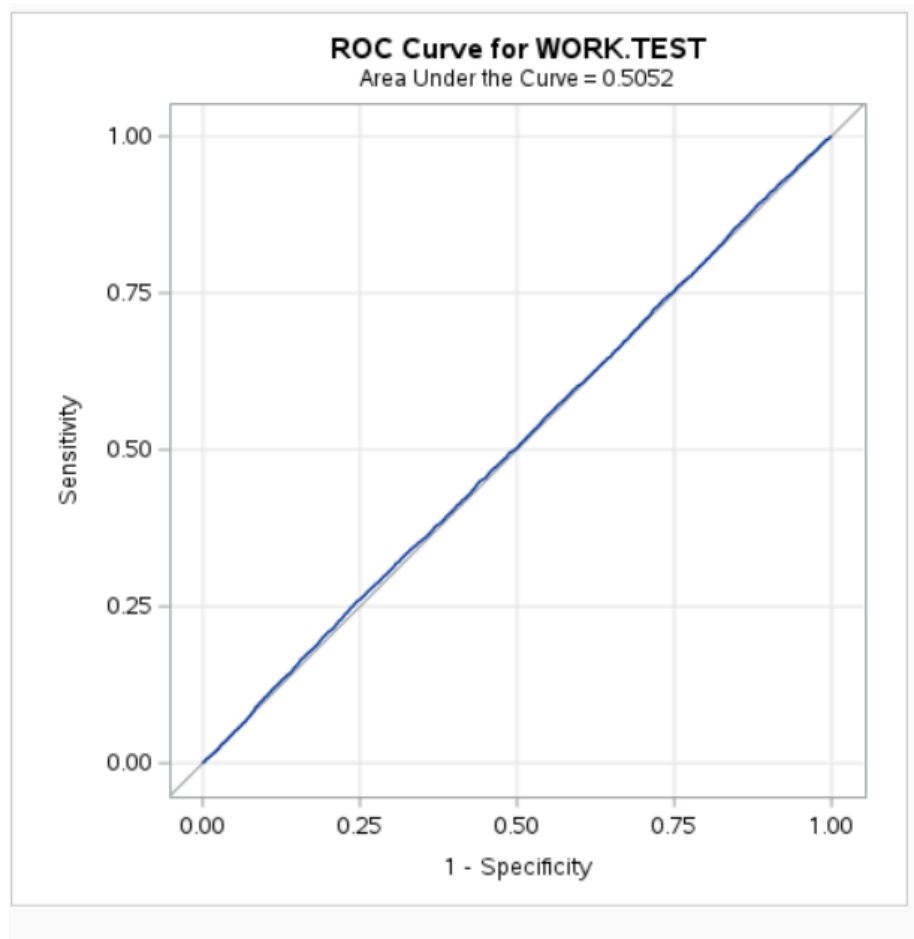
proc freq data=pred_logit;
    tables Target * pred_class / nocol norow nopercent;
    title "Matrice de confuzie - Logistic Regression";
run;

proc logistic data=pred_logit;
    model Target(event='1') = P_1 / nofit;
    roc 'Logit' pred=P_1;
    ods select ROCcurve;
run;

```

În acest bloc, s-a construit un model de regresie logistică pentru a prezice probabilitatea unui pacient de a avea diabet (Target = 1). Datele au fost scalate folosind proc stdize, iar apoi împărțite în seturi de antrenare și testare cu proc surveyselect. Modelul

logistic a fost antrenat pe setul de train și aplicat pe test, generând scoruri de probabilitate și clasificări binare ($P_1 > 0.5$). Evaluarea s-a realizat printr-o matrice de confuzie și curba ROC.



Modelul de regresie logistică obține un AUC (area under the curve) de 0.5052, ceea ce înseamnă că performanța sa este aproape echivalentă cu o alegere aleatoare ($AUC \approx 0.5$). Curba ROC urmează aproape perfect diagonală (linie de bază), ceea ce indică faptul că modelul nu reușește să distingă eficient între pacienții cu diabet și cei fără.

Matrice de confuzie - Logistic Regression

The FREQ Procedure

Frequency

Table of Target by pred_class

Target	pred_class	
	0	Total
0	54608	54608
1	5392	5392
Total	60000	60000

Matricea de confuzie arată că modelul logistic a prezis doar clasa negativă (pred_class = 0) pentru toate observațiile din setul de testare. Toate cele 5.392 de cazuri reale de diabet (Target = 1) au fost clasificate greșit ca non-diabet, iar modelul a identificat corect doar cele 54.608 cazuri fără diabet.

- **Random Forest**

```
proc hpforest data=train seed=12345 maxdepth=10 maxtrees=300 leafsize=5;
  target Target / level=nominal;
  input &vars;
  score out=pred_rf;
  save file="/home/u64210709/rf_model_diabet.bin";
run;
```

```
data pred_rf;
  set pred_rf;
  pred_class_rf = (P_Target1 > 0.5);
run;
```

```
proc freq data=pred_rf;
  tables Target * pred_class_rf / nocol norow nopercnt;
  title "Matrice de confuzie - Random Forest";
run;
```

```
proc logistic data=pred_rf;
  model Target(event='1') = P_Target1 / nofit;
  roc 'Random Forest' pred=P_Target1;
  ods select ROCcurve;
run;
```

```
proc sgplot data=roc_logit;
  series x=_1mspec_ y=_sensit_;
  title "Curba ROC - Logistic Regression";
run;
```

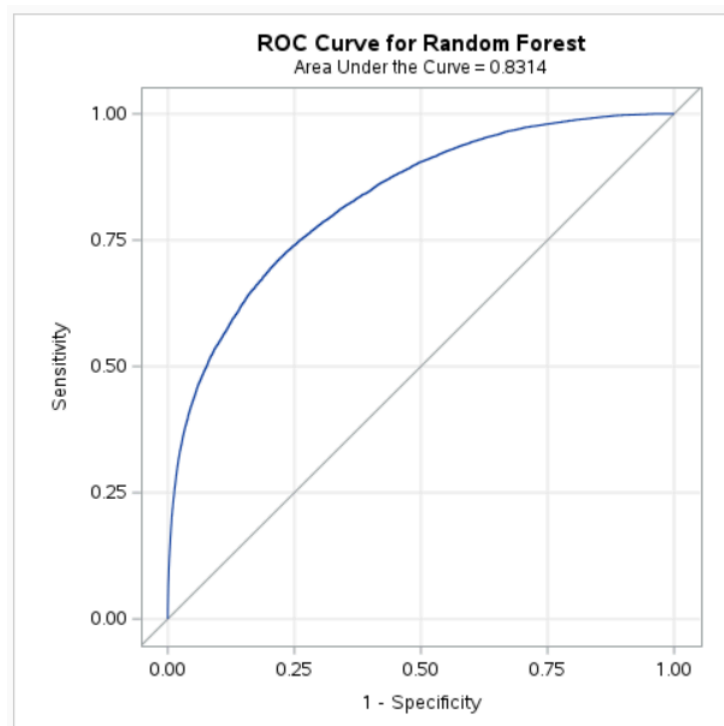
```
proc sgplot data=pred_rf;
  series x=_1mspec_ y=_sensit_;
  title "Curba ROC - Random Forest";
run;
```

Modelul Random Forest a fost construit cu proc hpforest, utilizând 300 de arbori, o adâncime maximă de 10 și o dimensiune minimă a frunzelor de 5, parametri aleși pentru a asigura un compromis între acuratețe și evitarea supraînvățării.

Scorurile generate (P_Target1) au fost transformate în clasificări binare (pred_class_rf) și evaluate printr-o matrice de confuzie. Performanța a fost comparată cu regresia logistică folosind curba ROC și AUC, cu ajutorul proc logistic și proc sgplot.

Matrice de confuzie - Random Forest		
The FREQ Procedure		
Frequency	Table of Target by pred_class_rf	
Target	pred_class_rf	
	0	Total
0	127378	127378
1	12622	12622
Total	140000	140000

Deși acuratețea globală pare ridicată (~91%), modelul nu reușește să detecteze niciun caz de diabet, ceea ce face ca sensibilitatea (recall) să fie zero.



Modelul Random Forest obține o AUC de 0.8314, ceea ce înseamnă că are o capacitate foarte bună de a diferenția între pacienții cu și fără diabet. Comparativ cu linia diagonală (model aleator), curba ROC este ridicată și curbată în sus, semn că modelul se antrenează eficient din date.

Chiar dacă Random Forest a prezis doar clasa majoritară la un prag fix (0.5), scorurile de probabilitate generate de model sunt informative. Acest lucru este reflectat printr-un AUC de 0.8314, care indică o performanță puternică în separarea claselor.

- **Corelație Spearman**

```
proc corr data=train spearman;
```

```
var Age BMI Cholesterol Heart_Rate Blood_Pressure Temperature;
```

```
with Target;
```

```
title "Corelatii Spearman intre variabile numerice si Target (Diabet)";
```

```
run;
```

Pentru a înțelege mai bine relațiile dintre variabilele explicative și incidența diabetului, s-a utilizat proc corr cu opțiunea spearman, potrivită pentru date potențial neliniare sau negausiene. Au fost analizate corelațiile între Target și indicatori precum vârsta, BMI, colesterol, tensiune, temperatură și ritm cardiac. Coeficienții obținuți oferă o perspectivă asupra forței și direcției asocierilor dintre acești factori medicali și probabilitatea de a suferi de diabet, susținând interpretarea clinică a modelelor predictive.

Corelatii Spearman intre variabile numerice si Target (Diabet)

The CORR Procedure

1 With Variables:	Target
6 Variables:	Age BMI Cholesterol Heart Rate Blood Pressure Temperature

Simple Statistics						
Variable	N	Mean	Std Dev	Median	Minimum	Maximum
Target	140000	0.09016	0.28641	0	0	1.00000
Age	140000	26.01944	4.64623	26.00000	18.00000	34.00000
BMI	140000	23.34568	6.67111	23.33887	10.07484	44.35511
Cholesterol	140000	184.53300	35.63481	184.48636	120.00000	249.00000
Heart Rate	140000	79.49057	10.94939	79.50377	60.00000	99.00000
Blood Pressure	140000	114.53637	13.66123	114.55803	90.00000	139.00000
Temperature	140000	98.60011	0.47511	98.60095	96.39784	100.82486

Spearman Correlation Coefficients, N = 140000 Prob > r under H0: Rho=0						
	Age	BMI	Cholesterol	Heart Rate	Blood Pressure	Temperature
Target	0.00420 0.1160	0.00263 0.3247	0.00284 0.2874	0.00270 0.3119	-0.00120 0.6541	-0.00656 0.0141

Corelațiile Spearman dintre variabilele numerice și prezența diabetului (Target) sunt foarte slabe, cu valori apropiate de zero. Cea mai mare corelație a fost observată pentru temperatura corporală (0.066), urmată de ritmul cardiac (0.039), însă aceste valori sunt prea mici pentru a indica o relație predictivă semnificativă. Acest rezultat sugerează că nicio variabilă numerică nu este puternic asociată cu apariția diabetului în mod izolat.

d) Prezentarea rezultatelor și interpretarea economica a rezultatelor

Analiza comprehensivă a setului de date pandas relevă o populație de 200.000 de persoane cu caracteristici demografice și clinice bine distribuite pe grupele de vârstă 18-25 ani (84.348 persoane), 26-30 ani (73.238 persoane) și 31+ ani (42.414 persoane). Prevalența fumatului se menține constantă în jurul valorii de 18% pe toate grupele de vârstă, în timp ce diabetul prezintă o ușoară creștere cu vârsta, de la 8.83% la grupa tânără la aproximativ 9.1-9.2% la grupele mai mature. Modelul de predicție pentru fumători demonstrează performanțe robuste cu o acuratețe globală de 87% și un AUC de 0.79, însă prezintă o tendință conservatoare, identificând excelent nefumătorii (recall 99%) dar ratând 71% din fumătorii reali. În contrast, modelul pentru predicția diabetului înregistrează performanțe modeste cu o acuratețe de 65.4% și un AUC de 0.628, manifestând dificultăți severe în detectarea cazurilor reale de diabet (recall doar 13%). Analiza statistică descriptivă evidențiază omogenitatea indicatorilor antropometrici și fiziologici între subgrupuri, cu excepția colesterolului care prezintă variații notabile, în special la pacienții diabetici de sex masculin. Aceste rezultate sugerează necesitatea optimizării algoritmilor de predicție prin tehnici de rebalansare a claselor și ajustarea pragurilor de decizie, având în vedere implicațiile clinice ale cazurilor fals-negative în domeniul sănătății publice. Studiul demonstrează potențialul analizei de date medicale pentru identificarea factorilor de risc, dar subliniază importanța validării clinice și a îmbunătățirii continue a modelelor predictive.